Manil Bastola
Stephanie Mertz
CSE 566S
Homework 3

All experiments in this paper were performed on Wustl Linux Lab server. The OpenMP experiments were conducted on single desktop running on vnc.q queue while the MPI experiments were conducted on multiple servers all running on all.q queue cluster.

1.) Compare and contrast the OpenMP and MPI versions using Scalasca. How much of the code's execution time is due to library overhead (thread/process management)? Which library is more efficient?

For this experiment we set the problem size n = 25 so that the execution overhead is comparable to the setup/communication overhead for both OpenMP and MPI. We repeated each experiment 4 times and averaged the results for statistical soundness. Each library (MPI and OpenMP) was analysed using scalasca for two different processes/threads size (2 and 16). This was done to analyse overheads for small vs large number of processes/threads for the two libraries.

For OpenMP with numThreads = 2, the total time of execution was 2.01 ± 0.08s with 99.0 ± 0.09 % of the total execution time is spent in Management i.e. omp parallel functions parallelizing different threads. The rest 0.63 ± 0.32% is spent on Synchronization i.e omp parallel functions joining the different threads.

With numThreads = 16, the total time of execution was 3.98 ± 0.10s with 78.64 ± 1.18 % of the total execution time is spent in Management i.e. omp parallel functions parallelizing different threads. The rest 19.1 ± 2.01% is spent on Synchronization i.e omp parallel functions joining the different threads.

For MPI, with numProcesses = 2, the total time of execution was 0.81 ± 0.23s with 91.35 ± 6.14 % of the total execution time is spent in MPI_Init function. The rest 7.4 ± 1.2% is spent on Collective Communication i.e. in MPI_Reduce function.

On the other hand, with numProcesses = 16, the total time of execution was 36.92 ± 2.16s with 99.7 ± 0.12% of the total execution time spent in MPI_Init (98.29 ± 2.46%) & MPI_Finalize (1.08 ± 0.09%) functions. The rest 0.26 ± 0.006% is spent in MPI_Reduce function. This is a large communication overhead compared to numProcesses = 2.

We found OpenMP to be far efficient than MPI. It seems as MPI is not at all that better than OpenMP, except that MPI can handle large data with its computers strung together (aka clusters). Since OpenMP uses threading to parallelize code, race conditions have to be carefully

avoided.However, MPI uses processes that only have private variables so this is not a concern with MPI.

The drawback to OpenMP is that performance is limited for a problem that scales up largely with problem size since the computational power is necessary to handle the problem size is limited by the number of cores in a processor. In scenarios where computational power is needed across processors, MPI can be used if the communication overhead can be amortized by the gains in cluster computing.

2.) What is the performance dependence upon the size of the problem? You will need to think a bit harder about how to adjust the problem size for this one (relative to earlier applications). Feel free to alter the code to generate new problem instances of a given size.

Boolean Circuit Satisfaction (CiruitSat) pertaining to k-CNFs are classes of problems of non-polynomial nature, especially when tackled in a brute-force manner. We hypothesize this problem for increasing problem size n, performance dependence is O(2^n). For example, the default code with n = 23, had a boolean array of size 23 each of which can take values 0 or 1. Any satisfiable condition needs to check 2^23 bits in the worst case.

For testing we modified the code as follows. The code was modified such that the circuit value depended on n:

```
int circuit_value ( int n, int bvec[] ){
        int value = 1;
        int i;
        for (i = 0; i < N-2; i+=2){
                value = value && (bvec[i] || bvec[i+1]) && (!bvec[i+1] || !bvec[i+2]) ;
        }
}
```

We kept the size of the processes (in case of MPI numProcesses = 16) or threads (in case of openMP, numThreads = 16) constant. The error bars were generated from student t-distribution. We observed the following results:

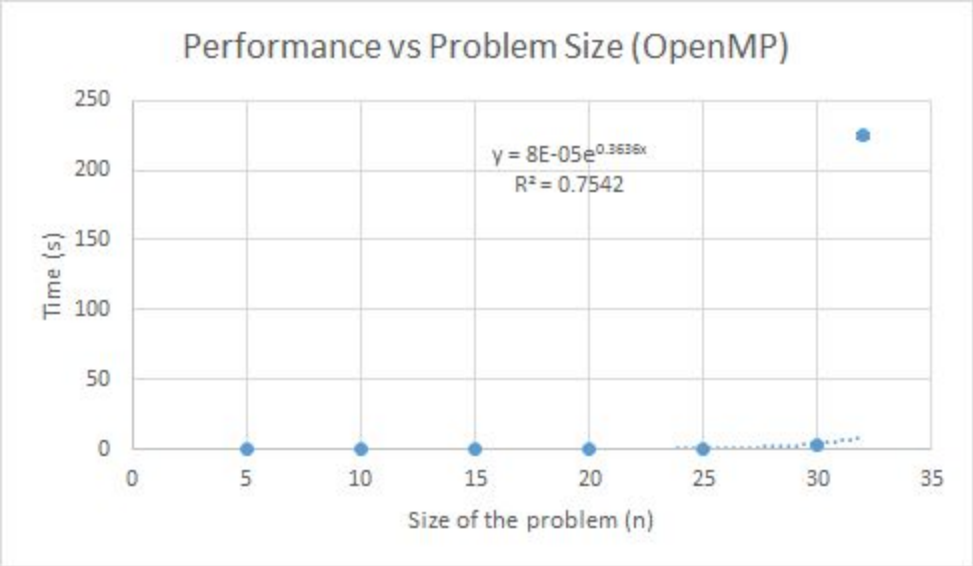


Figure 1: Performance vs Problem Size for Boolean Circuit Sat in OpenMP (numThreads= 16)

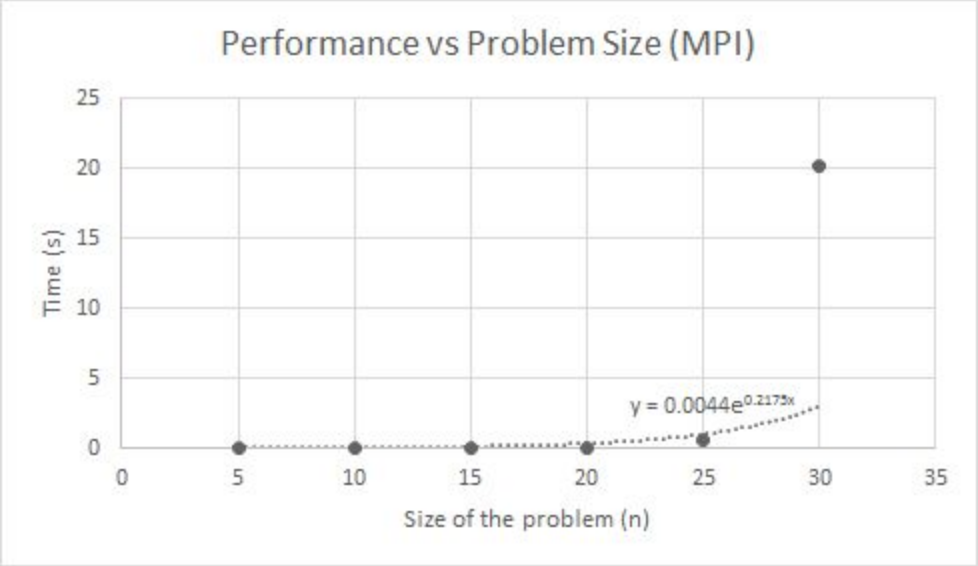
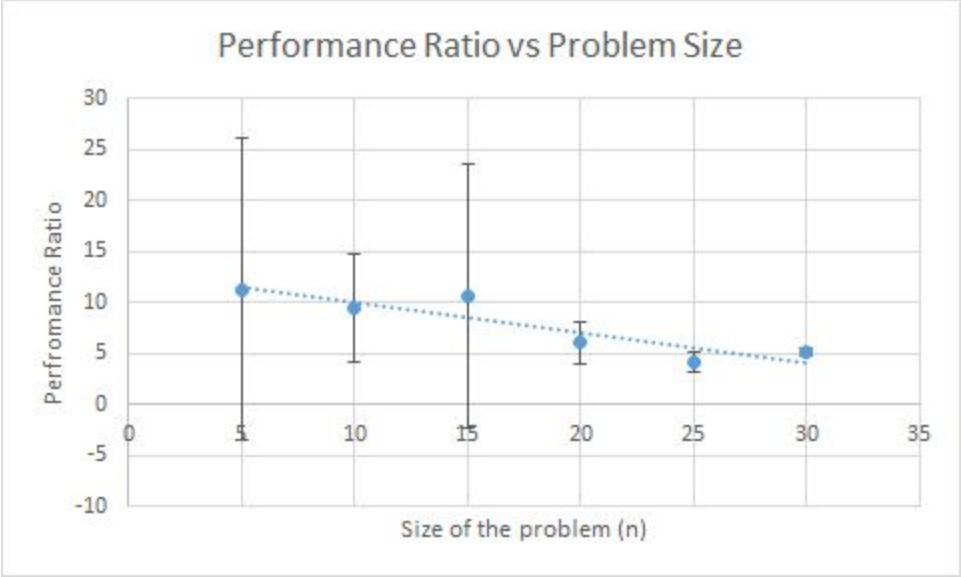


Figure 2: Performance vs Problem Size for Boolean Circuit Sat in MPI (numProcesses = 16)

One can observe that the trend is clearly exponential for both OpenMP and MPI. In excel, we were not able to generate curve fitting for exponents with base 2. With base *e*, the best fit line is performance t = 8E^-5*e*^(0.36n). We note that for equal exponents, any exponential function of one base is a constant multiple of an exponential function of another base (since log2(n) = ln(n)/ln(2) implies 2^(n) = Ce^(n)). Hence, the above model is O(2^n).

Additionally, in the case of MPI, we were able to use high-performance point-to-point messaging module by using the --mca btl ^openib flag. Despite this, we observe that the MPI version takes quite long compared to the OpenMP version. This was expected for smaller number processes and small problem size since the overhead related to communication in MPI is large while

OpenMP takes advantages of shared resources such as cache. Hence we hypothesized lowering of communication overhead with problem size for MPI. We plotted the performance ratio of MPI/OpenMP for increasing problem size below. We can see that, for increasing n, the communication overhead in MPI starts to taper down compared to OpenMP. The error bars were calculated from the standard error propagation method[1] utilizing the percentage errors in fig 1 and fig 2.

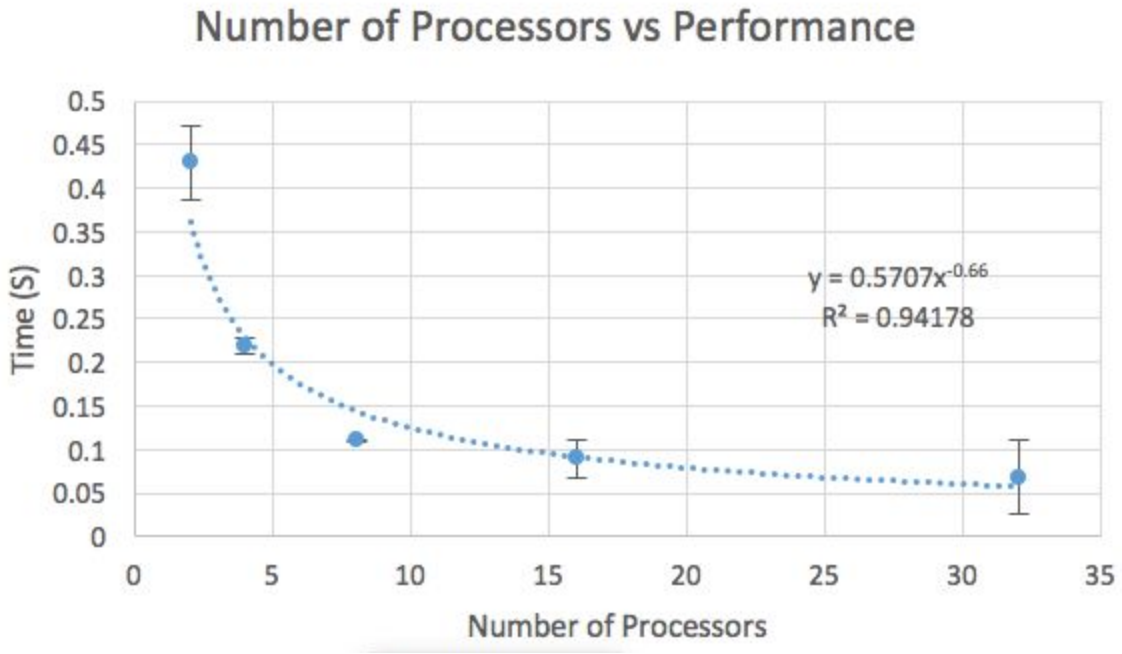


3.) How does the MPI version perform as the number of nodes (processes) is varied?

We hypothesized that because MPI would be able to paralyze the code across more cores on the cluster, we would see gains in performance beyond when the number of processors equaled the number of local cores, a restraining factor with OpenMP. To test this, we maintained a problem size of N = 23.

We observed consistent performance times with the data in #4 for up to 16 processes and an increase in performance for 32 processes. At 32 processes we did take notice that performance was plateauing due to time needed to reduce the results.

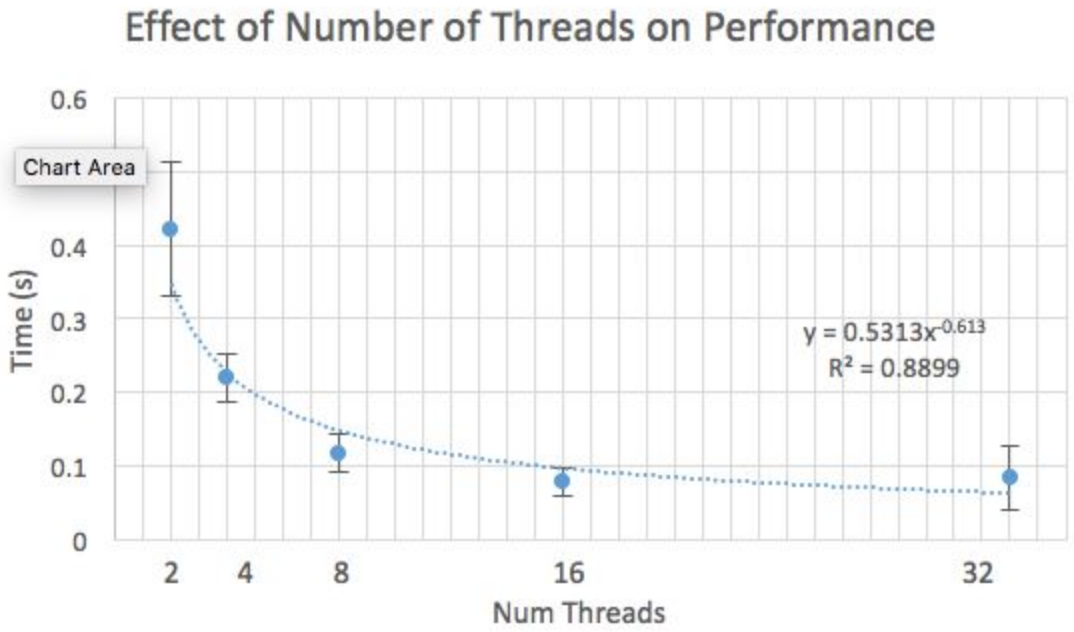
[1] A Summary of Error Propagation.. http://ipl.physics.harvard.edu/wp-uploads/2013/03/PS3_Error_Propagation_sp13.pdf



4.) How does the OpenMP version perform as the number of threads is varied?

CiruitSat is highly parallelizable code. For this problem, we used a problem size of N = 23. This experiment was run on the linux lab computers which has 16 available cores (8 digitalized). Due to this, we expected peak performance as numThreads = 16. Beyond this, we expect performance to plateau and degrade slightly. For any further performance gains, we would have to look to paralyzing the code across cores without shared memory using MPI.

We observed an inverse relationship between numThreads and performance as numThreads <= the number of processors. At 32 threads we noticed a slight decay in performance.



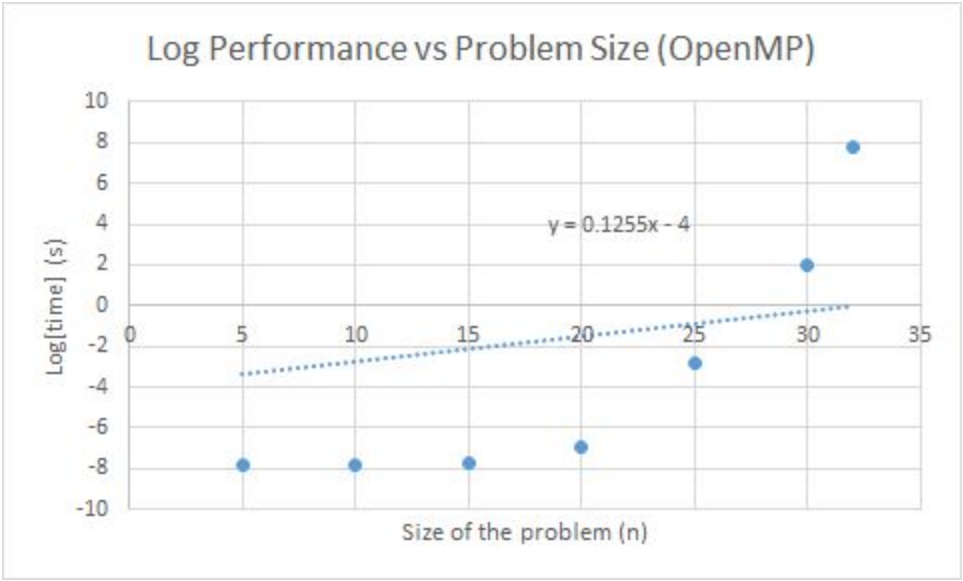
5.) Please put together a performance model that attempts to include as many of the above factors as is reasonable. For example, if you could define the performance of this code as a function based on problem size, number of threads/processes, etc., what would that function look like? Use the data you collected from previous problems to back up your answer to this problem.

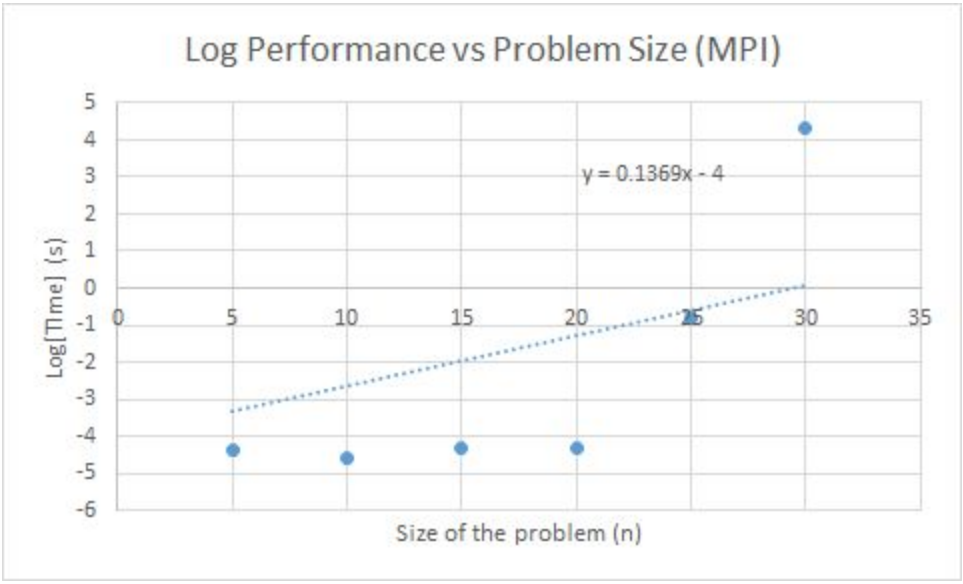
In problem 2, we saw that the running time is proportional to O(2^n) where n is the problem size. In 3 and 4 we observed that for OpenMP version of the code, the running time is inversely proportional to the number of threads as long as the number of threads is less than the number of cores. For MPI versions we expected the running time to be inversely proportional to number of processes for numProcesses > numCores (on a single machine) due to MPI's capability of utilizing cores on other cluster nodes. However we did not observe such trends. We observed somewhat linear trends till numProcesses <= 16. Hence, we hypothesize a constrained model where the performace (τ) of boolean CircuitSat is,

$\tau = C2^n/p$ where p = number of threads or processes. ($0 < p <= 16$ is region where linearity w.r.t. p holds)
and C is any offset due to program setup, communication overhead, etc.

$then,\ log_2\tau = nlog_2C - log_2p$

We modify figure 1 and 2 to express log performance vs. *n* to come up with the following plots:





In both cases, since p = 16, we fixed the y-intercept to -4 (i.e. $-log_2 16$) to compute the line of best. In OpenMP version, the best fit line has slope 0.1255. From that we find the value of constant term $C = 2^{0.1255} = 1.091$. Hence, the overhead due to "other" factors not considered in our model is fairly small for OpenMP. Similar is the case with MPI ($C = 1.10$). Finally, we note the limitations of the model for $0 < n < 32$ and $0 < p <= 16$.

Works Cited

Barney, Blaise. "OpenMP." OpenMP. N.p., n.d. Web. 06 Mar. 2017. Website

Ghildiyal, Pawan. "MPI vs OpenMP: A Short Introduction Plus Comparison (Parallel Computing)." MPI vs OpenMP: A Short Introduction Plus Comparison (Parallel Computing). N.p., 01 Jan. 1970. Web. 06 Mar. 2017. Website

"Open MPI Documentation." Open MPI Documentation. Software in the Public Interest, n.d. Web. 06 Mar. 2017.